UNIVERSITI TEKNOLOGI MARA
MARKING SCHEME

COURSE	:	COMPUTER ORGANIZATION AND ARCHITECTURE
COURSE CODE	:	CSC429
EXAMINATION	:	NOVEMBER 2026 (MOCK)
TIME	:	3 HOURS

MARKING SCHEME NOTES

1. This marking scheme accompanies the CSC429 mock paper (November 2026). Generated answers show working. Real-bank questions carry a labelled unofficial AI model answer, not an official scheme.

2. Marking points show the answer and mark allocation. Check model-generated real-bank points against the source paper.

3. Part A = 4 x 5 + 6 x 10; Part B = 1 x 20; total = 100 marks.

4. Generated slots: 2. Real-bank slots: 9. Unofficial model answers:

   9/9.
   For local study use only; model answers are unofficial.

5. Structure from 20 unique CSC429 past papers; every real question remains traceable to its source page.

FOR MARKERS' USE ONLY

This marking scheme consists of 9 printed pages

© Hak Cipta Universiti Teknologi MARA

PART A — MARKING GUIDE

QUESTION A1 (5 marks)

Topic: Parallel and multiprocessor architecture

Source: CSC429 (7).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Two advantages of SMP (Symmetric Multiprocessor) organization: (1) Performance/scalability — multiple processors share the same main memory and I/O, so several tasks (or threads) can execute in parallel, increasing throughput and allowing the system to scale by adding processors. (2) Cost-effectiveness and simplicity — all processors are identical and share one copy of the OS, memory and I/O resources, so there is no need to replicate memory or peripherals per processor, giving better price/performance and easier load balancing than equivalent multiple independent machines.

(b) Cluster organization: A cluster is a collection of independent, complete computers (nodes), each with its own processor and memory, interconnected by a high-speed network (e.g. LAN) and coordinated by clustering software so that they work together and appear to users as a single, unified, highly available system.

(c) One benefit of cluster organization: High availability / fault tolerance — if one node fails, the remaining nodes take over its workload, so the system continues to provide service (alternatively: scalability by adding nodes, or better price/performance using commodity machines).

Unofficial marking guide:

• 1 mark: SMP advantage 1 — parallel execution of tasks/threads on shared memory giving increased throughput/performance (or scalability by adding processors).

• 1 mark: SMP advantage 2 — cost-effectiveness/simplicity from identical processors sharing one memory, I/O and OS (no replication needed), or easier load balancing.

• 1 mark: cluster definition — a collection of independent computers/nodes.

• 1 mark: cluster definition — nodes interconnected by a high-speed network and coordinated by software to act as a single system (accept 'appears as one system' / 'work together').

• 1 mark: cluster benefit — any one valid benefit, e.g. high availability/fault tolerance, scalability, or price/performance using commodity nodes.

Model confidence: high.

Model note: Question text is clear; no OCR correction needed. For (c) any one valid benefit earns the mark; high availability is the most commonly expected answer.

QUESTION A2 (5 marks)

Topic: Instruction formats and addressing modes

Source: CSC429 (1).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The addressing technique illustrated in Figure 2 is direct addressing (also called absolute addressing). The instruction contains an opcode and an address field A, and the address field A points directly to the memory location that holds the operand — the effective address is simply EA = A, with no further indirection.

(b) How direct addressing is used: The instruction is fetched and decoded; the CPU takes the address field A from the instruction and uses it unchanged as the effective address of the operand. It then performs a single memory read at address A to fetch the operand into a register, after which the operation (e.g. ADD, LOAD) is executed on that operand. Because the operand address is embedded in the instruction itself, no index register, base register or pointer dereference is involved — the address in the instruction is the actual main-memory address of the data. This makes it suitable for referencing fixed, known memory locations such as named variables or memory-mapped I/O registers.

(c) Advantage: It is simple and fast — only one memory access is needed to fetch the operand (no extra indirection), and no special registers or address-calculation hardware are required. Disadvantage: The address field must be large enough to hold a full memory address, so the instruction is long (limited address space), and the address is fixed at assembly time, so the code is not relocatable and cannot easily support arrays, pointers or dynamic data structures.

Unofficial marking guide:

• 1 mark: Correctly labels the technique as direct (absolute) addressing.

© Hak Cipta Universiti Teknologi MARA

• 2 marks: Explains that the address field A is used directly as the effective address (EA = A) and that one memory access fetches the operand from that address (1 mark for EA = A / address used directly; 1 mark for the single operand fetch and execution).

• 2 marks: 1 mark for a valid advantage (e.g. simple, only one memory access, no special registers needed); 1 mark for a valid disadvantage (e.g. long instruction/large address field, limited address space, non-relocatable/fixed address, poor support for arrays and pointers).

Model confidence: high.

Model note: Figure 2 is inferred from the extracted text: an instruction with Opcode + Address A, and memory containing the operand at that address. This matches direct addressing; if the figure actually showed a pointer/indirect cell, the label would be indirect addressing. The question text refers to '9(a)' but the slot is A2 — treated as referring to part (a) of this question.

QUESTION A3 (5 marks)

Topic: Computer generations and history

Source: CSC429 (20).pdf, page(s) 2

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Inventors of the devices:
(i) Mechanical Calculating Machine — Blaise Pascal (the Pascaline, 1642). [Note: Charles Babbage's Difference Engine is also a mechanical calculating machine, but the standard CSC429 answer for the 'mechanical calculating machine' is Pascal; Gottfried Leibniz's stepped reckoner is a related later device.]
(ii) Analytical Machine (Analytical Engine) — Charles Babbage (designed c. 1837).
(iii) IAS Machine — John von Neumann (the IAS/Von Neumann machine, Institute for Advanced Study, Princeton, completed 1952).

(b) Later-generation machines are designed as general-purpose computers because:
- They use stored-program concepts (the von Neumann architecture), where both instructions and data reside in the same rewritable memory. The program can therefore be changed by loading new instructions rather than by rewiring or rebuilding the hardware, so one machine can solve many different problems.
- Advances in technology (transistors, integrated circuits, VLSI) made processing units fast, small, cheap and reliable enough to execute arbitrary instruction sets, so flexibility no longer costs much performance.
- General-purpose designs give greater flexibility, reusability and cost-effectiveness: a single machine serves many applications (scientific, business, multimedia), avoiding the expense of building a special-purpose machine for each task.
- Earlier machines (e.g. ENIAC, Pascal's calculator, Babbage's engines) were hard-wired or fixed-function, specific to one task, and had to be physically reconfigured to do anything else — slow, inflexible and expensive.
- General-purpose machines with an operating system and software layer support programmability, upgradability and a wide software ecosystem, improving overall efficiency and productivity.

Unofficial marking guide:

• 1 mark: (a)(i) Blaise Pascal named as inventor of the mechanical calculating machine

• 1 mark: (a)(ii) Charles Babbage named as inventor of the Analytical Engine/Machine

• 1 mark: (a)(iii) John von Neumann named as inventor of the IAS machine

• 1 mark: (b) explains stored-program concept / programmability — instructions and data in rewritable memory allow the same hardware to run different programs

• 1 mark: (b) explains flexibility/cost-effectiveness/reusability versus hard-wired special-purpose early machines, or the role of technology advances (transistors/ICs) enabling fast general-purpose designs

Model confidence: high.

Model note: OCR errors corrected conservatively: 'iiD' read as (iii); 'eadier' read as 'earlier'. For (a)(i), some texts credit Charles Babbage's Difference Engine as a mechanical calculating machine; the expected CSC429 answer is Blaise Pascal (Pascaline). If the official key expects Babbage for (i), accept that alternative. No figure/table is required for this question.

QUESTION A4 (5 marks)

Topic: Data representation and number systems

Answer: -6.7510 = 0xC0D80000 (sign=1, exponent=10000001, fraction=10110000000000000000000)

• 1 mark for the sign bit.

• 1 mark for the biased exponent.

• 1 mark for the 23-bit fraction.

• 1 mark for the correct 32-bit grouping.

• 1 mark for the final hexadecimal value.

© Hak Cipta Universiti Teknologi MARA

QUESTION A5 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (13).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Registers are small, high-speed storage locations inside the CPU used to hold data, instructions, addresses and control information during program execution. Their specific functions are:

Program Counter (PC): Holds the address of the next instruction to be fetched. It is automatically incremented (or updated by a branch) after each fetch so that the CPU knows where to continue execution.

Instruction Register (IR): Holds the instruction currently being executed. The instruction is fetched from memory into the IR, where it is decoded by the control unit to determine the operation and operands.

Memory Address Register (MAR): Holds the address of the memory location to be accessed (read or written). It connects the CPU to the memory address bus.

Memory Data Register (MDR) / Memory Buffer Register (MBR): Holds the data being transferred to or from memory. On a read, it receives the data from memory; on a write, it holds the data to be sent to memory. It acts as a buffer between the CPU and main memory.

Accumulator (AC): A general-purpose register that holds one operand for arithmetic and logic operations and often receives the result of those operations. It is central to the fetch–execute cycle in accumulator-based architectures.

General-Purpose Registers (GPRs): A set of registers (e.g., R0–R7) used by the programmer to hold operands, intermediate results, and addresses. They provide fast temporary storage and reduce the number of memory accesses.

Status/Flag Register (Program Status Word, PSW): Holds condition flags (zero, carry, sign, overflow) and control bits (interrupt enable, mode) that record the outcome of the last operation and the CPU state. It is used for conditional branching and system control.

Stack Pointer (SP): Holds the address of the top of the stack in memory. It is used for push/pop operations, subroutine calls and returns, and interrupt handling.

Index Register: Holds an offset value used in address calculation (indexed addressing) to access arrays or tables; it may be auto-incremented or auto-decremented.

Instruction Pointer / Base Register (where present): The instruction pointer is another name for the PC; a base register holds a base address for relocatable addressing. These are architecture-dependent but serve the same addressing and sequencing roles.

Unofficial marking guide:

- *1 mark: Correct definition of a register as small, fast CPU storage.*
- *1 mark: PC — holds address of next instruction; incremented after fetch.*
- *1 mark: IR — holds current instruction for decoding/execution.*
- *1 mark: MAR — holds memory address to be accessed.*
- *1 mark: MDR/MBR — holds data transferred to/from memory (buffer).*
- *1 mark: Accumulator — holds operand and/or result of ALU operations.*
- *1 mark: General-purpose registers — programmer-visible temporary data/address storage.*
- *1 mark: Status/Flag register — holds condition flags and control bits for branching/state.*
- *1 mark: Stack Pointer — points to top of stack for push/pop, calls, interrupts.*
- *1 mark: Index register — holds offset for indexed addressing (arrays/tables).*

Model confidence: high.

Model note: The question text has an OCR spacing error ('specificfunction'); corrected to 'specific function'. The question does not specify a fixed list of registers, so the answer covers the standard set; examiners should accept any correctly described CPU register types. No figure or table is required.

QUESTION A6 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (11).pdf, page(s) 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) The keyboard belongs to the human-readable / character-based (character I/O) category of I/O modules — specifically it is a character-oriented input device that exchanges data with the computer one

© Hak Cipta Universiti Teknologi MARA

character (byte) at a time. In the standard I/O module classification (human-readable, machine-readable, communication), the keyboard is a human-readable input device whose I/O module handles slow, character-by-character transfers and typically signals the CPU via an interrupt.

(b) Two other devices in the same human-readable/character-oriented category are: (1) the monitor/display (VDU) — a human-readable output device, and (2) the printer — a human-readable output device. (Other acceptable answers: mouse, touch screen, scanner — all human-readable devices.)

(c) The basic unit of exchange for the keyboard is the character (one byte / 8 bits), representing a single key code (e.g., in ASCII). Each keypress produces one character that is transferred to the I/O module's data register and then to the CPU.

(d) Two types of characters are: (1) Printable (graphic) characters — visible symbols such as letters (A–Z, a–z), digits (0–9), punctuation and special symbols; these are displayed or printed and are represented by codes such as ASCII 0x20–0x7E. (2) Control (non-printable) characters — codes that do not produce a visible symbol but control the device or data flow, e.g., carriage return (CR), line feed (LF), tab, backspace, escape (ESC), and delete; in ASCII these occupy codes 0x00–0x1F and 0x7F. The distinction matters because the I/O module must interpret control characters as commands (e.g., CR/LF move the cursor) while printable characters are stored/displayed as data.

Unofficial marking guide:

• 2 marks (a): 1 mark for identifying the human-readable/character-oriented category; 1 mark for explaining it is a character-based (byte-at-a-time) input device.

• 2 marks (b): 1 mark for each correct device in the same category (e.g., monitor, printer, mouse, scanner).

• 2 marks (c): 1 mark for stating the character; 1 mark for noting it is one byte/8 bits (e.g., ASCII code).

• 4 marks (d): 2 marks for each type of character — 1 mark for naming/defining the type (printable vs control) and 1 mark for a correct example/explanation of its role.

Model confidence: high.

Model note: The question does not specify a particular classification scheme; the answer assumes the standard human-readable / machine-readable / communication I/O category classification. Part (d) is interpreted as printable vs control characters, which is the most common reading of 'types of characters' in this context.

QUESTION A7 (10 marks)

Topic: Data representation and number systems

a) Answer: -11.37510 = 0xC1360000 (sign=1, exponent=10000010, fraction=01101100000000000000000)

• 1 mark for sign, exponent and mantissa.

• 1 mark for the biased exponent.

• 1 mark for the 23-bit fraction.

• 1 mark for the 32-bit binary grouping.

• 1 mark for the hexadecimal result.

b) Answer: 0x40500000 = 3.2510

• 1 mark for the sign bit.

• 1 mark for the exponent field.

• 1 mark for the fraction field.

• 1 mark for applying the bias and significand.

• 1 mark for the final decimal value.

QUESTION A8 (10 marks)

Topic: Input/output and interrupts

Source: CSC429 (3).pdf, page(s) 4

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Definition of interrupt [2 marks]

An interrupt is a signal (either generated by hardware or by software) sent to the CPU that temporarily suspends the normal sequential execution of the current program so that the processor can transfer control to a special routine — the interrupt service routine (ISR) or interrupt handler — which deals with the event that caused the interrupt. When the handler finishes, the CPU restores the saved context (program counter and register contents) and resumes the interrupted program as if nothing had happened. In short, an interrupt is an asynchronous (or synchronous, for traps) mechanism by which a

© Hak Cipta Universiti Teknologi MARA

device or program requests the CPU's attention.

(b) Two different types of interrupt [4 marks]

Type 1 — Hardware (external) interrupt: This is raised by a peripheral device or external signal, e.g. a keyboard key press, a disk controller signalling that a DMA transfer is complete, or a timer tick. It is asynchronous to the running program. Hardware interrupts are further classified as maskable (can be temporarily disabled by the CPU, e.g. an I/O completion) or non-maskable (cannot be disabled, e.g. a power-failure or memory-error signal).

Type 2 — Software interrupt (trap / exception): This is generated internally by the CPU as a result of executing an instruction or an exceptional condition, e.g. a system call (a program deliberately executing an instruction to request an OS service), a divide-by-zero, an invalid opcode, or a page fault. It is synchronous with the instruction stream and is often used to switch from user mode to supervisor mode.

(Other acceptable pairs: maskable vs non-maskable; vectored vs non-vectored; internal vs external; precise vs imprecise.)

(c) One advantage and one disadvantage of interrupts [4 marks]

Advantage — Improved CPU utilisation / responsiveness through concurrency: instead of the CPU busy-waiting (polling) a slow device, it can continue executing useful work and is only diverted when the device actually needs attention. This overlaps I/O with computation, raises throughput, and gives fast response to urgent events (e.g. a keystroke or a fault).

Disadvantage — Overhead and complexity: every interrupt involves saving/restoring the CPU context, running the ISR, and arbitrating priority, which consumes processor cycles and can degrade performance if interrupts arrive frequently (interrupt overhead). Interrupt handling also adds design complexity and introduces hazards such as priority inversion, lost or nested interrupts, and the need for careful synchronisation between the ISR and the main program.

Unofficial marking guide:

• 1 mark: states that an interrupt is a signal/event that diverts the CPU from its current program

• 1 mark: states that control passes to an interrupt service routine/handler and the interrupted program is later resumed (context saved/restored)

• 2 marks: correct description of a hardware/external interrupt with an example (e.g. keyboard, disk, timer)

• 2 marks: correct description of a software interrupt/trap/exception with an example (e.g. system call, divide-by-zero, page fault)

• 2 marks: valid advantage explained (e.g. avoids busy-waiting, overlaps I/O with computation, better CPU utilisation/responsiveness)

• 2 marks: valid disadvantage explained (e.g. context-save/restore overhead, complexity, priority/nesting problems)

Model confidence: high.

Model note: Question text is complete; no figure or table required. Marking assumes the standard 2+4+4 split given in the subparts. Any two genuinely distinct interrupt types (hardware vs software, maskable vs non-maskable, vectored vs non-vectored, internal vs external) are acceptable for part (b).

QUESTION A9 (10 marks)

Topic: Computer architecture and CPU organisation

Source: CSC429 (14).pdf, page(s) 3

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

The question asks for the function of each register in the computer system. In a typical von Neumann CPU, the main registers and their functions are: (1) Program Counter (PC): holds the address of the next instruction to be fetched; it is automatically incremented after each fetch so that instructions are executed in sequence unless a branch occurs. (2) Instruction Register (IR): holds the instruction currently being executed; its opcode is decoded by the control unit to generate control signals. (3) Memory Address Register (MAR): holds the address of the memory location to be accessed (read or written); it connects the CPU to the address bus. (4) Memory Data Register (MDR) / Memory Buffer Register (MBR): holds the data read from or to be written to memory; it acts as a buffer between the CPU and the data bus. (5) Accumulator (AC): a general-purpose register that holds one operand and the result of arithmetic/logic operations performed by the ALU. (6) Status/Flag Register (PSW): holds condition flags (e.g., zero, carry, sign, overflow) and control bits that reflect the outcome of the last ALU operation and the CPU state. (7) General-Purpose Registers (GPRs): hold operands, intermediate results, and addresses during program execution; they provide fast temporary storage. (8) Stack Pointer (SP): holds the address of the top of the stack in memory, used for subroutine calls, returns, and interrupt handling. (9) Index/Base Registers: hold base or index values used in address calculation for indexed or base-relative addressing. (10)

© Hak Cipta Universiti Teknologi MARA

Instruction Pointer (IP) is another name for the PC in x86 terminology. Each register is a small, high-speed storage location inside the CPU; together they hold the address, data, and control information needed to fetch, decode, and execute instructions.

Unofficial marking guide:

• 1 mark: Correctly identifies and describes the Program Counter (PC) as holding the address of the next instruction.
• 1 mark: Correctly identifies and describes the Instruction Register (IR) as holding the current instruction.
• 1 mark: Correctly identifies and describes the Memory Address Register (MAR) as holding a memory address.
• 1 mark: Correctly identifies and describes the Memory Data Register (MDR/MBR) as holding data read/written to memory.
• 1 mark: Correctly identifies and describes the Accumulator (AC) as holding operands/results for the ALU.
• 1 mark: Correctly identifies and describes the Status/Flag Register (PSW) as holding condition flags.
• 1 mark: Correctly identifies and describes General-Purpose Registers (GPRs) as holding operands/intermediate results.
• 1 mark: Correctly identifies and describes the Stack Pointer (SP) as pointing to the top of the stack.
• 1 mark: Correctly identifies and describes Index/Base registers for address calculation.
• 1 mark: Overall clarity and correct terminology (e.g., registers are fast CPU storage; address/data/control roles).
Model confidence: high.
Model note: The question text does not specify a particular CPU architecture; the answer assumes a generic von Neumann CPU with common registers. Some architectures may use different names (e.g., IP for PC, MBR for MDR) or omit some registers; equivalent names should be accepted. No figure or table was provided, so the answer is based on standard computer organization knowledge.

QUESTION A10 (10 marks)

Topic: Memory hierarchy and cache

Source: CSC429 (12).pdf, page(s) 4, 5

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

(a) Access methods for the storage devices/memory in the scenarios:
(i) Cassette tape (Ahmad's tape): Sequential access — data is read in order along the tape; to reach a later track the tape must be wound forward past earlier tracks.
(ii) Floppy disk (National Archive resource): Direct access — the disk is organised into tracks and sectors, so the drive can seek directly to a track/sector without reading everything before it (random/direct access to blocks).
(iii) Cache memory (Sara's specification): Random access — any cache line can be accessed directly in essentially constant time, independent of address order.
(iv) RAM (Hana's game console requirement): Random access — any memory location can be read or written directly in constant time.

(b) SRAM (Static RAM): built from flip-flop/latch circuits (typically 4–6 transistors per bit), so it retains its stored value as long as power is supplied and needs no refresh. It is very fast, has low access time, but is expensive and has low density, so it is used for cache memory and registers.
DRAM (Dynamic RAM): stores each bit as charge on a capacitor (one transistor plus capacitor per cell), so the charge leaks away and the cell must be periodically refreshed (typically every few milliseconds). It is slower than SRAM, cheaper and denser, so it is used for main memory. DRAM also uses destructive read-out (the value is sensed and then rewritten).

(c) In a three-level cache architecture (L1, L2, L3), L1 is the fastest cache level; it is smallest and closest to the CPU core. Cache is implemented using SRAM (static RAM), because SRAM is fast and does not require refreshing.

Unofficial marking guide:

• 1 mark: (a)(i) cassette tape — sequential access
• 1 mark: (a)(ii) floppy disk — direct access
• 1 mark: (a)(iii) cache — random access
• 1 mark: (a)(iv) RAM — random access
• 1 mark: (b) SRAM built from flip-flops/latches, no refresh needed, fast/expensive/low density
• 1 mark: (b) SRAM used for cache/registers
• 1 mark: (b) DRAM stores charge on capacitors, needs periodic refresh, slower/cheaper/denser
• 1 mark: (b) DRAM used for main memory
• 1 mark: (c) L1 is the fastest cache level
• 1 mark: (c) cache is implemented using SRAM
Model confidence: high.

© Hak Cipta Universiti Teknologi MARA

Model note: The question text lists scenarios but the subpart (a) refers to items (i)–(iv) that are not explicitly numbered in the extracted text. I assumed the four devices/memory are: (i) cassette tape, (ii) floppy disk, (iii) cache, (iv) RAM, in the order they appear in the scenarios. If the original paper numbered them differently, map the access methods accordingly. No figure or table is required.

© Hak Cipta Universiti Teknologi MARA

PART B — MARKING GUIDE

QUESTION B1 (20 marks)

Topic: Assembly-language programming

Source: CSC429 (18).pdf, page(s) 6

Unofficial model answer — generated by an AI model, not an official UTM marking scheme.

Assumption: Figure 2 is interpreted as y = A - ((2C * D) / (B + A^2)). The expression is evaluated as: temp1 = 2*C; temp2 = temp1*D; temp3 = A*A; temp4 = B + temp3; temp5 = temp2 / temp4; y = A - temp5. Registers A, B, C, D are fixed; x, y, z are temporary. Final result must be in y.

One-address program (accumulator-based):

```
LOAD A       ; ACC = A
MUL A        ; ACC = A^2
ADD B        ; ACC = A^2 + B
STOR z       ; z = A^2 + B
LOAD C       ; ACC = C
MUL D        ; ACC = C*D
ADD C        ; ACC = 2*C*D  (since C*D + C*D = 2*C*D)
DIV z        ; ACC = (2*C*D)/(A^2+B)
STOR x       ; x = (2*C*D)/(A^2+B)
LOAD A       ; ACC = A
SUB x        ; ACC = A - x
STOR y       ; y = final answer
```

Two-address program (operand1 = operand1 op operand2):

```
MOV x, C     ; x = C
ADD x, C     ; x = 2C
MUL x, D     ; x = 2C*D
MOV z, A     ; z = A
MUL z, A     ; z = A^2
ADD z, B     ; z = B + A^2
DIV x, z     ; x = (2C*D)/(B + A^2)
MOV y, A     ; y = A
SUB y, x     ; y = A - x
```

Three-address program (dest = src1 op src2):

```
MUL x, C, D   ; x = C*D
ADD x, x, x   ; x = 2*C*D
MUL z, A, A   ; z = A^2
ADD z, B, z   ; z = B + A^2
DIV x, x, z   ; x = (2*C*D)/(B + A^2)
SUB y, A, x   ; y = A - x
```

Unofficial marking guide:

- *1 mark: correct interpretation of Figure 2 as y = A - ((2C*D)/(B + A^2)) and correct order of operations.*
- *6 marks: one-address program – 1 mark for correct use of LOAD/STOR, 1 mark for computing A^2+B, 1 mark for computing 2C*D, 1 mark for division, 1 mark for final subtraction, 1 mark for storing final answer in y.*
- *6 marks: two-address program – 1 mark for MOV to initialise, 1 mark for computing 2C, 1 mark for 2C*D, 1 mark for A^2+B, 1 mark for division, 1 mark for final subtraction into y.*
- *6 marks: three-address program – 1 mark for MUL x,C,D, 1 mark for doubling to 2C*D, 1 mark for MUL z,A,A, 1 mark for ADD z,B,z, 1 mark for DIV x,x,z, 1 mark for SUB y,A,x.*
- *1 mark: correct assembly notation (mnemonics and operand order) consistent with Table 1 for all three programs.*

Model confidence: medium.

*Model note: Figure 2 is not fully legible from the extracted text; the expression is assumed to be y = A - (2C*D)/(B + A^2). If the figure shows a different grouping (e.g., denominator B + A^2 or (B+A)^2), the programs must be adjusted accordingly. Table 1 uses inconsistent punctuation (e.g., ADD (p.q.r)); operand order is taken as dest, src1, src2 for three-address and dest, src for two-address. One-address instructions are assumed to use an implicit accumulator.*

END OF MARKING SCHEME

© Hak Cipta Universiti Teknologi MARA